Batch: A2 Roll No.: 16010324044
Experiment / assignment / tutorial No. 5
Grade: AA / AB / BB / BC / CC / CD /DD

Signature of the Staff In-charge with date

TITLE: Deadlock

AIM: Implementation of Deadlock Avoidance Banker's Algorithm

OUTCOME: Student can Compare different algorithms used for management and scheduling of processes.

Implementation of Deadlock Avoidance Banker's Algorithm.

Also Prove that the safety algorithm requires an order of $m \times n^2$ operations.

Write full code for all algorithms along with output and screen shot.

Algorithm:

1. Calculate Need = Max - Allocation.

2. Set Work = Available.

3. Set Finish[i] = false for every process.

4. Find an i such that:

   Finish[i] == false

   Need[i] <= Work

5. If such i exists:

   Work = Work + Allocation[i]

   Finish[i] = true

   Add Pi to Safe Sequence

   Go to Step 4.

6. If all Finish[i] == true:

    System is SAFE.

7. Otherwise:

    System is UNSAFE.

Proof:

Let n be the number of processes and m be the number of resources.

Checking one process requires comparing its Need with Work for m resources → O(m).

In the worst case, all n processes are checked in each pass.

Up to n passes may be required.

Therefore,

$O(n * n * m) = O(mn^2)$

Hence, the Safety Algorithm requires $O(m \times n^2)$ operations in the worst case.

Code:

```c
#include <stdio.h>


int main()

{

    int n, m;


    printf("Enter number of processes: ");

    scanf("%d", &n);


    printf("Enter number of resource types: ");

    scanf("%d", &m);


    int Allocation[n][m];
```

```
int Max[n][m];

int Need[n][m];

int Available[m];


printf("\nEnter Allocation Matrix:\n");

for (int i = 0; i < n; i++)

{

    printf("P%d: ", i);

    for (int j = 0; j < m; j++)

    {

        scanf("%d", &Allocation[i][j]);

    }

}


printf("\nEnter Max Matrix:\n");

for (int i = 0; i < n; i++)

{

    printf("P%d: ", i);

    for (int j = 0; j < m; j++)

    {

        scanf("%d", &Max[i][j]);

    }

}


printf("\nEnter Available Resources:\n");

for (int j = 0; j < m; j++)
```

```
    {
        scanf("%d", &Available[j]);
    }

    /* Calculate Need Matrix */
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < m; j++)
        {
            Need[i][j] = Max[i][j] - Allocation[i][j];
        }
    }

    /* Display Need Matrix */
    printf("\nNeed Matrix:\n");

    for (int i = 0; i < n; i++)
    {
        printf("P%d: ", i);

        for (int j = 0; j < m; j++)
        {
            printf("%d ", Need[i][j]);
        }

        printf("\n");
```

```
}


int Work[m];


for (int j = 0; j < m; j++)

{

    Work[j] = Available[j];

}


int Finish[n];


for (int i = 0; i < n; i++)

{

    Finish[i] = 0;

}


int SafeSequence[n];

int count = 0;


/* Safety Algorithm */


while (count < n)

{

    int found = 0;


    for (int i = 0; i < n; i++)
```

```
    {
        if (Finish[i] == 0)
        {
            int possible = 1;

            for (int j = 0; j < m; j++)
            {
                if (Need[i][j] > Work[j])
                {
                    possible = 0;
                    break;
                }
            }

            if (possible)
            {
                for (int j = 0; j < m; j++)
                {
                    Work[j] += Allocation[i][j];
                }

                SafeSequence[count] = i;
                count++;

                Finish[i] = 1;
                found = 1;
```

```
            }

        }

    }


    if (found == 0)

    {

        break;

    }

}


/* Check Safe State */


if (count == n)

{

    printf("\nSystem is in SAFE STATE.\n");


    printf("Safe Sequence: ");


    for (int i = 0; i < n; i++)

    {

        printf("P%d", SafeSequence[i]);


        if (i != n - 1)

            printf(" -> ");

    }
```

```
        printf("\n");

    }

    else

    {

        printf("\nSystem is in UNSAFE STATE.\n");

    }



    return 0;

}
```

Output:

```
Enter number of processes: 5
Enter number of resource types: 3

Enter Allocation Matrix:
P0: 0 1 0
P1: 2 0 0
P2: 3 0 2
P3: 2 1 1
P4: 0 0 2

Enter Max Matrix:
P0: 7 5 3
P1: 3 2 2
P2: 9 0 2
P3: 2 2 2
P4: 4 3 3

Enter Available Resources:
3 3 2

Need Matrix:
P0: 7 4 3
P1: 1 2 2
P2: 6 0 0
P3: 0 1 1
P4: 4 3 1

System is in SAFE STATE.
Safe Sequence: P1 -> P3 -> P4 -> P0 -> P2
```

Conclusion:

The Banker's Algorithm for deadlock avoidance was successfully implemented and tested. The Need matrix was calculated using the Maximum and Allocation matrices, and the Safety Algorithm was used to determine whether the system was in a safe state. For the given input, a safe sequence P1 → P3 → P4 → P0 → P2 was obtained, confirming that the system is in a

safe state. The time complexity of the Safety Algorithm was also analyzed and found to be O(m × n^2) in the worst case.

Signature of faculty in-charge